《数据库系统原理》实验报告（6）					
题目：miniob 进阶实验					
学号	2453359	姓名	李欣洋	日期	5.10
实验环境：miniob					

实验步骤及结果截图：

1.drop table 整体思路

整体链路：解析 -> 语义构造 -> 执行 -> 资源回收 -> 元数据删除

1. 词法/语法先识别 DROP TABLE，把它解析成 SCF_DROP_TABLE。
2. Stmt::create_stmt() 根据 SCF_DROP_TABLE 生成 DropTableStmt。
3. CommandExecutor 再把 StmtType::DROP_TABLE 分发给 DropTableExecutor。
4. DropTableExecutor 调 Db::drop_table()，让数据库层统一处理删除逻辑。
5. Db::drop_table() 找到表后调用 Table::destroy()，由表对象负责 sync、释放引擎、删除 .table/.data/.lob/.index 等文件。

核心代码片段：

src/observer/sql/stmt/stmt.cpp

```
case StmtType::DROP_TABLE: {
  return DropTableStmt::create(db, sql_node.drop_table, stmt);
}
```

src/observer/sql/executor/command_executor.cpp

```
case StmtType::DROP_TABLE: {
  DropTableExecutor executor;
  rc = executor.execute(sql_event);
} break;
```

src/observer/sql/executor/drop_table_executor.cpp

```
RC DropTableExecutor::execute(SQLStageEvent *sql_event)
{
  RC rc = RC::SUCCESS;
  DropTableStmt *drop_table_stmt = dynamic_cast<DropTableStmt *>(sql_event->stmt());
  if (nullptr == drop_table_stmt) {
    return RC::INVALID_ARGUMENT;
  }

  Db *db = sql_event->session_event()->get_current_db();
  if (nullptr == db) {
    return RC::SCHEMA_DB_NOT_OPEN;
  }

  rc = db->drop_table(drop_table_stmt->table_name().c_str());
  return rc;
}
```

src/observer/storage/db/db.cpp

```
RC Db::drop_table(const char *table_name)
{
  Table *table = find_table(table_name);
  if (nullptr == table) {
    return RC::SCHEMA_TABLE_NOT_EXIST;
  }

  RC rc = table->destroy(path_.c_str());
  if (rc != RC::SUCCESS) {
    return rc;
  }

  tables_.erase(table_name);
  delete table;
  return RC::SUCCESS;
}
```

src/observer/storage/table/table.cpp

```
RC Table::destroy(const char *base_dir)
{
  if (engine_ != nullptr) {
    engine_->sync();
    engine_.reset();
  }

  std::string table_meta_file = std::string(base_dir) + "/" + meta_name_;
  std::string data_file       = std::string(base_dir) + "/" + data_name_;

  if (0 != ::remove(table_meta_file.c_str())) {
    return RC::IOERR_REMOVE;
  }
  if (0 != ::remove(data_file.c_str())) {
    return RC::IOERR_REMOVE;
  }

  return RC::SUCCESS;
}
```

2.Date 整体思路

整体链路：新增类型 -> 识别字面量 -> 入库/比较 -> 输出格式化 -> 校验合法性

1. **词法层新增 DATE_T 和 DATE_STR，让 date 关键字和 'YYYY-MM-DD' 字面量都能被识别。**
2. **语法层把 DATE_T 映射成 AttrType::DATES，让建表时可以声明日期列。**
3. **value 规则支持 DATE_STR，解析时就先校验日期是否合法，不合法直接报错。**
4. **新增 DateType，负责日期比较、字符串化、以及从字符串转换成内部整数存储。**

5. Value 和 DataType 里补上 DATES 支持，保证插入、查询、load data、比较、输出都能走通。

核心代码片段：

```
src/observer/sql/parser/yacc_sql.y

|DATE_STR {
  $$ = new Value();
  if (!value_init_date($$, $1)) {
    yyerror(&@1, sql_string, sql_result, scanner, "invalid date");
    YYERROR;
  }
}

check_date(int y, int m, int d)
{
  static const int mon[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
  const bool leap = (y % 400 == 0 || (y % 100 != 0 && y % 4 == 0));
  return y > 0 && m > 0 && m <= 12 && d > 0 && d <= mon[m] + ((m == 2 && leap) ? 1 : 0);
}

src/observer/common/type/date_type.cpp

RC DateType::set_value_from_str(Value &val, const string &data) const
{
  int date_value = 0;
  if (!parse_date_string(data, date_value)) {
    return RC::SCHEMA_FIELD_TYPE_MISMATCH;
  }
  val.set_int(date_value);
  val.set_type(AttrType::DATES);
  return RC::SUCCESS;
}

RC DateType::to_string(const Value &val, string &result) const
{
  const int date = val.value_.int_value_;
  char buf[16] = {0};
  snprintf(buf, sizeof(buf), "%04d-%02d-%02d", date / 10000, (date % 10000) / 100, date % 100);
  result = buf;
  return RC::SUCCESS;
}

src/observer/common/value.cpp

case AttrType::DATES: {
  value_.int_value_ = *(int *)data;
  length_           = length;
} break;

case AttrType::DATES: {
  set_int(value.get_int());
  set_type(AttrType::DATES);
} break;
```

使用的测试案例：

Date:

-- 1. 创建表并且插入合法数据

CREATE TABLE date_case_1(id int, d date);

INSERT INTO date_case_1 VALUES (1,'2020-01-01');

INSERT INTO date_case_1 VALUES (2,'2020-01-09');

INSERT INTO date_case_1 VALUES (3,'2020-12-31');

SELECT * FROM date_case_1;

```
miniob > CREATE TABLE date_case_1(id int, d date);
SUCCESS
miniob >   INSERT INTO date_case_1 VALUES (1,'2020-01-01');
SUCCESS
miniob >   INSERT INTO date_case_1 VALUES (2,'2020-01-09');
SUCCESS
miniob >   INSERT INTO date_case_1 VALUES (3,'2020-12-31');
SUCCESS
miniob >   SELECT * FROM date_case_1;
id | d
1 | 2020-01-01
2 | 2020-01-09
3 | 2020-12-31
```

-- 2. 插入边界数据

CREATE TABLE date_case_2(id int, d date);

```
INSERT INTO date_case_2 VALUES (1,'2019-12-31');
INSERT INTO date_case_2 VALUES (2,'2020-01-01');
INSERT INTO date_case_2 VALUES (3,'2020-02-29');
INSERT INTO date_case_2 VALUES (4,'2021-01-01');
```

```
miniob > CREATE TABLE date_case_2(id int, d date);
SUCCESS
miniob >   INSERT INTO date_case_2 VALUES (1,'2019-12-31');
SUCCESS
miniob >   INSERT INTO date_case_2 VALUES (2,'2020-01-01');
SUCCESS
miniob >   INSERT INTO date_case_2 VALUES (3,'2020-02-29');
SUCCESS
miniob >   INSERT INTO date_case_2 VALUES (4,'2021-01-01');
SUCCESS
```

```
-- 3. 删除一个元组
CREATE TABLE date_case_3(id int, d date);
INSERT INTO date_case_3 VALUES (1,'2018-01-01');
INSERT INTO date_case_3 VALUES (2,'2018-02-01');
DELETE FROM date_case_3 WHERE d < '2018-12-31';
SELECT * FROM date_case_3;
```

```
miniob > CREATE TABLE date_case_3(id int, d date);
SUCCESS
miniob >   INSERT INTO date_case_3 VALUES (1,'2018-01-01');
SUCCESS
miniob >   INSERT INTO date_case_3 VALUES (2,'2018-02-01');
SUCCESS
miniob >   DELETE FROM date_case_3 WHERE d < '2018-12-31';
SUCCESS
miniob >   SELECT * FROM date_case_3;
id | d
```

```
-- 4. 创建数据索引
CREATE TABLE date_case_4(id int, d date);
CREATE INDEX idx_date_case_4 ON date_case_4(d);
INSERT INTO date_case_4 VALUES (1,'2022-01-01');
INSERT INTO date_case_4 VALUES (2,'2022-06-15');
INSERT INTO date_case_4 VALUES (3,'2023-01-01');
```

```
miniob > CREATE TABLE date_case_4(id int, d date);
SUCCESS
miniob >   CREATE INDEX idx_date_case_4 ON date_case_4(d);
SUCCESS
miniob >   INSERT INTO date_case_4 VALUES (1,'2022-01-01');
SUCCESS
miniob >   INSERT INTO date_case_4 VALUES (2,'2022-06-15');
SUCCESS
miniob >   INSERT INTO date_case_4 VALUES (3,'2023-01-01');
SUCCESS
```

```
-- 5. 非法的日期应该报错
CREATE TABLE date_case_5(id int, d date);
INSERT INTO date_case_5 VALUES (1,'2017-02-29');
INSERT INTO date_case_5 VALUES (2,'2017-13-01');
INSERT INTO date_case_5 VALUES (3,'2017-11-31');
INSERT INTO date_case_5 VALUES (4,'2017-00-10');
```

```
miniob > CREATE TABLE date_case_5(id int, d date);
SUCCESS
miniob >   INSERT INTO date_case_5 VALUES (1,'2017-02-29');
SQL_SYNTAX > Failed to parse sql
miniob >   INSERT INTO date_case_5 VALUES (2,'2017-13-01');
FAILURE
miniob >   INSERT INTO date_case_5 VALUES (3,'2017-11-31');
SQL_SYNTAX > Failed to parse sql
miniob >   INSERT INTO date_case_5 VALUES (4,'2017-00-10');
FAILURE
```

Drop Table:

-- 1. 删除空表

```
CREATE TABLE drop_case_1(id int, name char);
DROP TABLE drop_case_1;
```

```
miniob > CREATE TABLE drop_case_1(id int, name char);
SUCCESS
miniob >   DROP TABLE drop_case_1;
SUCCESS
```

-- 2. 删除非空表

```
CREATE TABLE drop_case_2(id int, name char);
INSERT INTO drop_case_2 VALUES (1,'OB');
INSERT INTO drop_case_2 VALUES (2,'OceanBase');
DROP TABLE drop_case_2;
```

```
miniob > CREATE TABLE drop_case_2(id int, name char);
SUCCESS
miniob >   INSERT INTO drop_case_2 VALUES (1,'OB');
SUCCESS
miniob >   INSERT INTO drop_case_2 VALUES (2,'OceanBase');
SUCCESS
miniob >   DROP TABLE drop_case_2;
SUCCESS
```

-- 3. 删除之后重建表

```
CREATE TABLE drop_case_3(id int, name char);
INSERT INTO drop_case_3 VALUES (1,'bef');
DROP TABLE drop_case_3;
CREATE TABLE drop_case_3(id int, name char);
INSERT INTO drop_case_3 VALUES (2,'aft');
SELECT * FROM drop_case_3;
```

```
miniob >   CREATE TABLE drop_case_3(id int, name char);
SUCCESS
miniob >   INSERT INTO drop_case_3 VALUES (1,'bef');
SUCCESS
miniob >   DROP TABLE drop_case_3;
SUCCESS
miniob >   CREATE TABLE drop_case_3(id int, name char);
SUCCESS
miniob >   INSERT INTO drop_case_3 VALUES (2,'aft');
SUCCESS
miniob >   SELECT * FROM drop_case_3;
id | name
2 | aft
```

-- 4. 删除带索引的表

```
CREATE TABLE drop_case_4(id int, name char);
CREATE INDEX idx_drop_case_4 ON drop_case_4(id);
INSERT INTO drop_case_4 VALUES (1,'A');
INSERT INTO drop_case_4 VALUES (2,'B');
DROP TABLE drop_case_4;
```

```
miniob > CREATE TABLE drop_case_4(id int, name char);
SUCCESS
miniob >   CREATE INDEX idx_drop_case_4 ON drop_case_4(id);
SUCCESS
miniob >   INSERT INTO drop_case_4 VALUES (1,'A');
SUCCESS
miniob >   INSERT INTO drop_case_4 VALUES (2,'B');
SUCCESS
miniob >   DROP TABLE drop_case_4;
SUCCESS
```

-- 5. 删除一个不存在的表

DROP TABLE drop_case_5;

CREATE TABLE drop_case_5(id int, name char);

DROP TABLE drop_case_5;

DROP TABLE drop_case_5;

```
miniob > DROP TABLE drop_case_5;
FAILURE
miniob >   CREATE TABLE drop_case_5(id int, name char);

SUCCESS
miniob >   DROP TABLE drop_case_5;
SUCCESS
miniob >   DROP TABLE drop_case_5;
FAILURE
miniob >
```

出现的问题：

解决方案：